

MCDI500

Evaluación Sumativa 4 — Fase 4

Proyecto final integrador: análisis, reproducibilidad y comunicación de resultados

Informe técnico grupal · 30 % (+10 % autoevaluación y coevaluación)

Título del proyecto: Calidad del aire en Santiago — Detección de episodios críticos de MP2.5 mediante algoritmos eficientes, POO y ciencia de datos reproducible (Proyecto final integrador F1–F4)

Integrantes: Rodrigo Chinchón Ayala · Sergio Fernández Almonacid · Pablo Villalobos González

Curso: MCDI500 — Herramientas de Software Científico (202681.2535)

Docente: Dr. Omar Salinas Silva

Institución: Universidad Andrés Bello — Facultad de Ingeniería

Repositorio: github.com/Rfcha/abp_cienciadatos (F4-Entrega-Final)

https://github.com/Rfcha/abp_cienciadatos.git

Fecha: 21/06/2026

ÍNDICE

ÍNDICE	2
II. Introducción y contextualización	3
III. Definición de la problemática y objetivos del proyecto	3
IV. Aplicación de herramientas científicas	4
a. Reproducibilidad técnica	4
b. Control de versiones	4
c. Documentación del proceso	4
d. Arquitectura	5
e. Repositorio GitHub actualizado	5
V. Diseño de soluciones algorítmicas eficientes	5
a. Codificación funcional y arquitectura básica del script	5
b. Preprocesamiento y transformación del dataset	6
c. Validación técnica y verificación del código	6
d. Eficiencia y optimización	6
e. Diseño estructurado del código	7
f. Notebook final ejecutado y documentado	7
VI. Implementación de código modular y robusto (POO)	8
a. Programación orientada a objetos	8
b. Documentación de arquitectura	9
VII. Construcción de visualizaciones de datos efectivas	9
a. Preprocesamiento y transformación de datos	9
b. Visualizaciones analíticas	9
VIII. Metodología del desarrollo técnico	11
a. Validación y reflexión técnica	11
b. Resultados y Hallazgos del Análisis	11
c. Discusión — Limitaciones, Riesgos y Alcance	12
d. Conclusiones — Aprendizajes y Trabajo Futuro	13
e. Trazabilidad de mejoras	13
IX. Presentación audiovisual	14
X. Bibliografía (APA 7)	14
Anexo A. Evidencia de control de versiones (F1–F4)	15
Anexo B. Arquitectura y estructura del proyecto	19

II. Introducción y contextualización

La contaminación atmosférica por material particulado fino (MP2.5) constituye uno de los principales riesgos ambientales para la salud pública en la Región Metropolitana de Santiago. Durante los meses fríos, las condiciones de estabilidad atmosférica favorecen la acumulación de contaminantes, generando episodios críticos recurrentes que afectan a la población. Comprender, medir y caracterizar estos episodios de forma reproducible es el eje del presente proyecto.

Este informe constituye la síntesis integradora del Proyecto ABP desarrollado a lo largo del curso MCDI500, consolidando las cuatro fases del Aprendizaje Basado en Proyectos: la definición del problema y el entorno reproducible (F1), la obtención, limpieza y transformación de los datos (F2), la implementación del núcleo algorítmico con eficiencia y POO (F3), y la presente fase de análisis, reproducibilidad y comunicación de resultados (F4).

Objetivo general. Consolidar un sistema reproducible de análisis de calidad del aire en Santiago que detecte y caracterice episodios críticos de MP2.5, integrando programación estructurada, recursiva y orientada a objetos sobre datos oficiales de la red SINCA, y comunicar sus resultados de forma profesional, trazable y reproducible.

Justificación técnica. El problema requiere un enfoque basado en programación y análisis de datos porque involucra un volumen masivo de registros horarios (192.720), la detección de patrones temporales no triviales y la necesidad de algoritmos eficientes para localizar y ordenar concentraciones críticas. La combinación de recursividad de complejidad óptima, vectorización y una arquitectura orientada a objetos extensible permite abordar estas exigencias de forma rigurosa y reproducible.

Conexión con las fases F1–F4. Cada fase contribuye a la solución: F1 establece el entorno reproducible y la definición del problema; F2 produce el dataset consolidado y limpio; F3 implementa el núcleo algorítmico que detecta y caracteriza los episodios; y F4 integra los resultados, valida la reproducibilidad de extremo a extremo y comunica los hallazgos.

III. Definición de la problemática y objetivos del proyecto

Formulación de la problemática. ¿Bajo qué condiciones meteorológicas y temporales se producen los episodios críticos de MP2.5 en Santiago, y cómo pueden detectarse y ordenarse de forma eficiente sobre un gran volumen de datos horarios? El fenómeno a analizar es la superación del umbral operativo de 50 $\mu\text{g}/\text{m}^3$ de MP2.5 y su asociación con variables meteorológicas y contextuales.

Preguntas centrales del análisis:

- ¿Qué variables meteorológicas se asocian con mayor fuerza a las concentraciones elevadas de MP2.5?
- ¿Qué condiciones contextuales (inversión térmica, fin de semana, festivos) modifican las concentraciones?

¿Cómo localizar y ordenar eficientemente los episodios críticos sobre ~192.720 registros?

Objetivos específicos:

- Construir funciones especializadas que transformen la serie de MP2.5 en episodios críticos, con contratos claros y validación defensiva.
- Implementar algoritmos recursivos de búsqueda binaria $O(\log n)$ y merge sort $O(n \log n)$.
- Ejecutar mediciones reproducibles de complejidad con timeit y comparar implementaciones alternativas.
- Aplicar los pilares de la POO —encapsulamiento, herencia y polimorfismo— sobre una clase base abstracta con método plantilla.
- Documentar la arquitectura, asegurar la trazabilidad mediante control de versiones profesional y comunicar los resultados.

Alcance y supuestos. El análisis se circunscribe al dataset horario de la red SINCA (período 2022–2023, 11 estaciones de la Región Metropolitana, 192.720 registros). Se asume que el umbral operativo $MP2.5 \geq 50 \mu g/m^3$ es representativo del criterio crítico, y que el dataset consolidado en F2 es íntegro y trazable. El proyecto no aborda modelamiento predictivo, reservado como trabajo futuro.

IV. Aplicación de herramientas científicas

El proyecto se desarrolla íntegramente sobre el ecosistema científico de Python, integrando NumPy y Pandas para el procesamiento de datos, Matplotlib y Seaborn para la visualización, Jupyter para la ejecución reproducible y Git/GitHub para el control de versiones colaborativo.

a. Reproducibilidad técnica

El entorno se fija mediante un archivo requirements.txt versionado y un ambiente virtual aislado. El proyecto se ejecuta de extremo a extremo sin errores mediante «Kernel → Restart & Run All». El stack principal queda registrado con versiones explícitas:

Componente	Versión	Rol en el proyecto
Python	3.12.10	Lenguaje base del proyecto
pandas	2.3.3	Manipulación y consolidación de datos
numpy	2.3.5	Operaciones vectorizadas y arreglos
matplotlib	3.10.0	Visualización analítica
jupyterlab	4.4.5	Ejecución reproducible de notebooks

Tabla 1. Componentes, versiones y su rol en el proyecto.

Comandos de reproducibilidad:

```
python -m venv .venv
.venv\Scripts\Activate.ps1 # Windows / PowerShell
pip install -r requirements.txt
jupyter lab # Kernel -> Restart & Run All
```

b. Control de versiones

El proyecto utiliza Git y GitHub de forma sistemática, con commits descriptivos siguiendo la convención Conventional Commits (feat, fix, docs, refactor, chore), ramas tipadas (feature/*, fix/*, docs/*) e integración exclusiva mediante Pull Requests sobre la rama main protegida. Las identidades de los tres integrantes se consolidan mediante un archivo .mailmap, garantizando trazabilidad individual sin reescribir el historial.

La trazabilidad colaborativa se auditó sobre el historial completo del repositorio **abp_cienciadatos**, con las identidades de los tres integrantes consolidadas mediante **.mailmap**. El panel de evidencia arroja 226 commits 100 % trazables, 81 Pull Requests (72 fusionados, merge rate 88,9 %), 36 merges de integración y cero issues abiertos, con un ciclo promedio de revisión de 12,9 minutos. La participación individual se distribuye en Rodrigo Chinchón Ayala 166 commits (73,5 %), Sergio Fernández Almonacid 33 (14,6 %) y Pablo Villalobos González 27 (11,9 %). El detalle gráfico completo —dashboard ejecutivo, matriz por fase, participación por integrante, evolución diaria, mapa de calor y modelo de trazabilidad— se presenta en el **Anexo A** (Figuras A1–A6).

c. Documentación del proceso

Cada decisión técnica relevante queda registrada en las celdas narrativas markdown de los notebooks, en el README técnico del repositorio y en el archivo changelog.md, que documenta para cada mejora la fecha, descripción, commit asociado y justificación técnica. Esta triple documentación asegura coherencia entre el informe, el repositorio y los notebooks.

d. Arquitectura

La arquitectura del proyecto se organiza como un pipeline reproducible de cinco etapas —datos, preprocesamiento, análisis exploratorio, algoritmos y resultados— sustentado en una capa transversal de reproducibilidad y colaboración (notebooks documentados, control de versiones y revisión por Pull Request). El flujo de extremo a extremo, desde la ingesta de los datos oficiales de la red SINCA hasta la generación de hallazgos para la toma de decisiones, y el ecosistema tecnológico que lo soporta (Python, Jupyter, Pandas, NumPy, Matplotlib y Git/GitHub) se presentan en el diagrama del Anexo B (Figura B1).

e. Repositorio GitHub actualizado

El repositorio organiza el trabajo en carpetas por fase (F1, F2, F3-calidad-aire-santiago, F4-entrega-final), con un README técnico actualizado, archivo de dependencias y evidencia de contribuciones individuales. El esquema de la estructura del proyecto y su resumen ejecutivo por fases se presenta en el Anexo B (Figura B2).

Contribuciones individuales. La participación de cada integrante, auditada sobre el historial de Git con identidades consolidadas vía .mailmap, se documenta de forma detallada en el panel de control de versiones del apartado IV.b (Figuras 1–6). En síntesis, sobre los 226 commits trazables del repositorio, Rodrigo Chinchón Ayala concentra 166 (73,5 %), Sergio Fernández Almonacid 33 (14,6 %) y Pablo Villalobos González 27 (11,9 %), evidencia objetiva de la contribución individual exigida por la rúbrica.

La verificación es reproducible mediante los siguientes comandos, cuyos resultados se exportan a la carpeta de evidencias:

```
git shortlog -s -n --all          # commits por integrante (166 / 33 / 27)
git rev-list --all --count        # total de commits (226)
git rev-list --all --merges --count # merges de integración (36)
gh pr list --state all           # Pull Requests (81: 72 merged, 9 closed)
```

V. Diseño de soluciones algorítmicas eficientes

Este apartado presenta el desarrollo técnico del núcleo, evidenciando la construcción de soluciones algorítmicas eficientes, modulares y bien estructuradas. El núcleo se implementa en el notebook 02_algoritmos.ipynb.

a. Codificación funcional y arquitectura básica del script

La solución adopta una arquitectura modular que separa responsabilidades en tres capas: carga y preprocesamiento mínimo, codificación funcional que produce los episodios críticos, y algoritmos recursivos de búsqueda y ordenamiento. Cada función expone parámetros tipados, valida sus entradas y retorna estructuras verificables.

Función	Responsabilidad
detectar_episodios()	Agrupar rachas de horas consecutivas sobre el umbral y retorna episodios (inicio, fin, máximo).
busqueda_binaria_recursiva()	Localiza recursivamente el primer valor que supera el umbral en la serie ordenada — $O(\log n)$.
merge_sort_recursivo()	Ordena las concentraciones de MP2.5 mediante divide y vencerás — $O(n \log n)$.
contar_criticos_vectorizado()	Conteo de horas críticas mediante operación vectorizada de NumPy.
resumen()	Método plantilla de la clase base: estadísticas generales por contaminante.

Tabla 2. Funciones principales del núcleo algorítmico y sus responsabilidades.

b. Preprocesamiento y transformación del dataset

El núcleo consume el dataset consolidado en F2 (192.720 registros horarios) y aplica las transformaciones necesarias para el análisis. La caracterización estadística de las variables principales se resume a continuación:

Variable	Reg. válidos	Media	Mediana	Mínimo	Máximo
MP2.5 ($\mu\text{g}/\text{m}^3$)	189.830	26.20	24.40	2.00	97.40
MP10 ($\mu\text{g}/\text{m}^3$)	189.830	54.51	50.80	5.40	184.00
Temperatura ($^{\circ}\text{C}$)	192.720	14.00	14.00	-4.00	35.00
Humedad (%)	189.830	62.17	62.20	21.40	100.00
Viento (m/s)	192.720	3.10	2.90	0.20	15.80
Radiación (W/m^2)	189.830	306.99	43.00	0.00	1417.0

Tabla 3. Estadística descriptiva del dataset consolidado (notebook 01_exploracion.ipynb).

El umbral crítico se operacionaliza como $\text{MP2.5} \geq 50 \mu\text{g}/\text{m}^3$, coherente con los criterios trabajados en F2. Bajo este umbral se detectaron 4.918 episodios críticos de MP2.5 (8.579 horas críticas acumuladas), mientras que MP10 prácticamente no registró episodios en el período.

c. Validación técnica y verificación del código

La validación se ejecuta de forma sistemática sobre tres escenarios, dejando evidencia en celdas ejecutadas del notebook: casos normales (detección y búsqueda sobre la serie real), casos límite (series vacías, umbral igual a cero, ausencia de cruces) y excepciones (tipos no numéricos y umbrales no positivos disparan ValueError con mensajes explícitos).

Validación	Resultado esperado
Merge Sort vs sorted() de Python	Idéntico ✓
Búsqueda binaria vs barrido lineal	Mismo índice ✓
Serie vacía / umbral ≤ 0	ValueError controlado
Consistencia de tipos	Correcta

Tabla 4. Matriz de validación técnica del núcleo algorítmico.

d. Eficiencia y optimización

El rendimiento se midió con la biblioteca estándar timeit (100 repeticiones por implementación, promediadas). Se compararon dos pares de implementaciones que aíslan dos dimensiones de optimización: búsqueda lineal $O(n)$ vs búsqueda binaria recursiva $O(\log n)$, y bucle Python $O(n)$ vs vectorización NumPy $O(n)$. Antes de medir se verificó que cada par produce resultados idénticos, de modo que la comparación de tiempos sea legítima. Las mediciones se resumen a continuación:

Comparación (100 reps., $n = 70.368$)	Tiempo A (ms)	Tiempo B (ms)	Mejora
Búsqueda lineal $O(n) \rightarrow$ binaria recursiva $O(\log n)$	3,1768	0,0017	$\approx 1.852\times$
Conteo: bucle Python $O(n) \rightarrow$ vectorización NumPy $O(n)$	2,7703	0,0376	$\approx 74\times$

Tabla 5. Tiempos medidos con timeit (promedio de 100 repeticiones) y mejora relativa de cada optimización.

Interpretación en términos de Big-O. En la búsqueda, la mejora de $\sim 1.852\times$ refleja el salto de $O(n)$ a $O(\log n)$: sobre datos ordenados, la búsqueda binaria localiza el umbral en ≈ 17 comparaciones ($\lceil \log_2 70.368 \rceil$) frente al barrido lineal completo. En el conteo, ambas implementaciones son $O(n)$ y la ganancia de $\sim 74\times$ no proviene de la complejidad asintótica sino del menor factor constante de NumPy, que ejecuta la operación vectorizada en código compilado en lugar del bucle interpretado de Python. Ambos pares fueron validados como equivalentes (mismo índice de cruce y mismo conteo de 8.579 horas críticas) antes de comparar sus tiempos.

Impacto de la complejidad algorítmica: búsqueda lineal $O(n)$ vs. búsqueda binaria $O(\log n)$

Ejemplo con 192.720 registros ordenados



Figura 1. Costo asintótico: la búsqueda binaria $O(\log n)$ escala logarítmicamente frente a la lineal $O(n)$.

Para $n = 192.720$ registros, la búsqueda binaria basta con ~17 comparaciones frente a un barrido potencialmente completo de la búsqueda lineal. En el conteo masivo, ambas implementaciones comparten complejidad $O(n)$, pero la vectorización NumPy reduce drásticamente el tiempo por su menor factor constante al operar en código compilado.

En el conteo masivo de horas críticas, la vectorización con NumPy operó sobre código compilado y evitó el recorrido explícito en Python, lo que se tradujo en una mejora sustancial del tiempo de ejecución a igual complejidad $O(n)$ respecto del bucle interpretado. Estas mediciones, junto con la comparación de búsqueda lineal frente a binaria recursiva, se obtuvieron con timeit y son reproducibles en el notebook 02_algoritmos.ipynb.

Justificación de la elección final. Para la localización del umbral se adopta la búsqueda binaria recursiva por su complejidad $O(\log n)$ sobre datos ordenados; para el conteo masivo se adopta la vectorización NumPy por su menor factor constante. Ambas decisiones se fundamentan en mediciones reproducibles y en el análisis conjunto de complejidad temporal y espacial.

e. Diseño estructurado del código

La arquitectura organiza los componentes con alta cohesión y bajo acoplamiento. El flujo de datos es unidireccional y verificable: el DataFrame validado alimenta la capa funcional, que produce episodios y series ordenadas, consumidas por los algoritmos recursivos y las clases analizadoras. La estructura es escalable: incorporar un nuevo contaminante consiste en crear una subclase de AnalizadorBase, sin alterar el resto del sistema.

f. Notebook final ejecutado y documentado

El entregable central lo constituyen dos notebooks ejecutables y reproducibles, ejecutados de extremo a extremo mediante «Restart & Run All»: 01_exploracion.ipynb (pipeline de F2 y análisis exploratorio) y 02_algoritmos.ipynb (núcleo de F3: codificación funcional, algoritmos recursivos, análisis de complejidad, POO y validación). Cada notebook combina código eficiente y probado con celdas narrativas markdown que documentan las decisiones técnicas, manteniendo correspondencia total con el informe.

VI. Implementación de código modular y robusto (POO)

a. Programación orientada a objetos

El núcleo aplica los tres pilares de la POO sobre una clase base abstracta que define el contrato común del sistema:

- Encapsulamiento: la serie y el umbral se almacenan como atributos protegidos, accedidos mediante métodos que validan el estado.
- Herencia: AnalizadorMP25 y AnalizadorMP10 heredan de AnalizadorBase (ABC), reutilizando el comportamiento común.
- Polimorfismo: el método abstracto nombre_contaminante() se redefine en cada subclase; el método plantilla resumen() opera de forma uniforme sobre cualquier analizador concreto.

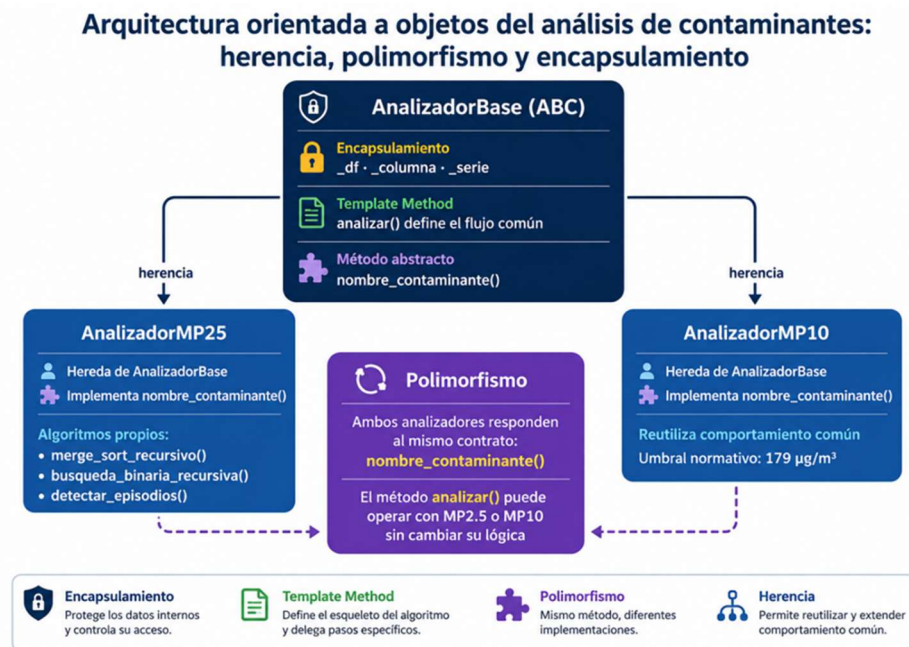


Figura 2. Arquitectura orientada a objetos del núcleo: clase base abstracta, herencia, polimorfismo y método plantilla (elaboración propia). El umbral de referencia para MP10 es $150 \mu\text{g}/\text{m}^3$, conforme a la implementación del núcleo y a la Tabla 6.

```
from abc import ABC, abstractmethod

class AnalizadorBase(ABC):
    """Contrato común para analizadores de contaminantes."""
    def __init__(self, serie: pd.Series, umbral: float):
        self._serie = serie          # encapsulamiento
        self._umbral = umbral

    @abstractmethod
    def nombre_contaminante(self) -> str: ... # polimorfismo
    def resumen(self) -> dict:                # Template Method
        return {'contaminante': self.nombre_contaminante(),
                'umbral': self._umbral,
                'total_registros': int(self._serie.size)}

class AnalizadorMP25(AnalizadorBase):        # herencia
    def nombre_contaminante(self) -> str:
        return 'MP2.5'
```

Listado 1. Clase base abstracta con encapsulamiento, polimorfismo y método plantilla.

El polimorfismo se evidencia al invocar resumen() de forma uniforme sobre instancias de distinto tipo, que producen un resultado especializado por contaminante:

Contaminante	Umbral	Registros	Media	Horas crít.	% crítico
MP2.5	50.0	70.368	26.21	8.579	12.19 %
MP10	150.0	70.368	54.54	149	0.21 %

Tabla 6. Salida polimórfica del método resumen () para cada analizador.

b. Documentación de arquitectura

La sección 9 del notebook documenta componentes, responsabilidades y principios de diseño. El patrón aplicado es Template Method: el método resumen() de la base define el esqueleto del cálculo estadístico y delega en nombre_contaminante() el paso variable, alineándose con el principio abierto/cerrado.

Componente	Responsabilidad principal
AnalizadorBase (ABC)	Contrato común, encapsulamiento, estadísticas generales y método plantilla resumen().
AnalizadorMP25	Análisis especializado de MP2.5, algoritmos recursivos y detección de episodios.
AnalizadorMP10	Análisis especializado de MP10 reutilizando el comportamiento heredado.
Capa funcional	Detección, conteo y normalización, desacoplada de la presentación.

Tabla 7. Componentes del núcleo y distribución de responsabilidades.

VII. Construcción de visualizaciones de datos efectivas

a. Preprocesamiento y transformación de datos

El dataset consolidado en F2 se somete a limpieza coherente (manejo de NA, casting de tipos y normalización de variables), quedando listo para la visualización. Las visualizaciones siguientes se construyen sobre el dataset procesado de 189.830 registros válidos de MP2.5.

b. Visualizaciones analíticas

Visualización 1 — Correlación con variables meteorológicas. La matriz de correlación de Pearson revela una asociación inversa dominante de MP2.5 con la temperatura (−0.54) y la radiación (−0.40), y directa con la humedad (+0.30), coherente con la fenomenología de los episodios invernales en Santiago.

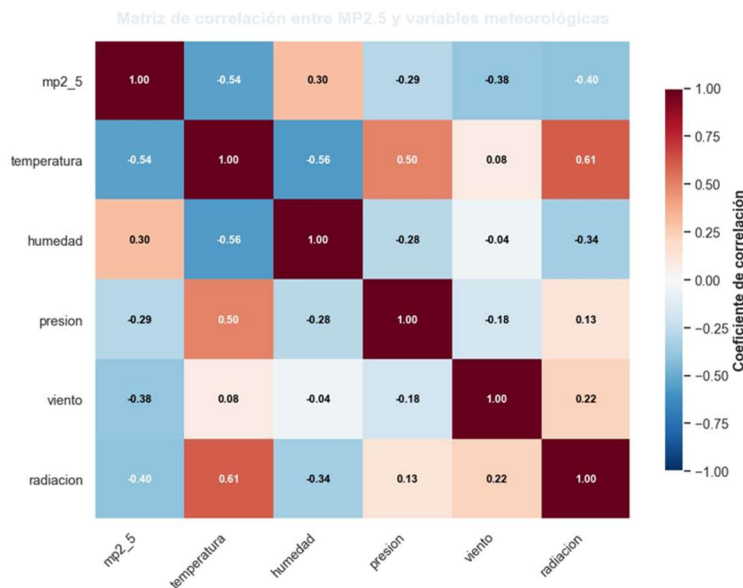


Figura 3. Matriz de correlación entre MP2.5 y las variables meteorológicas (fuente: SINCA 2022–2023, elaboración propia).

Visualización 2 — Condiciones asociadas a episodios críticos. El análisis por variables booleanas confirma que la inversión térmica es la condición más asociada a concentraciones elevadas (+6.21 $\mu\text{g}/\text{m}^3$ de diferencia media). En cambio, los fines de semana (−5.39) y festivos (−5.66) presentan menores concentraciones, consistente con la reducción de actividad y tránsito.

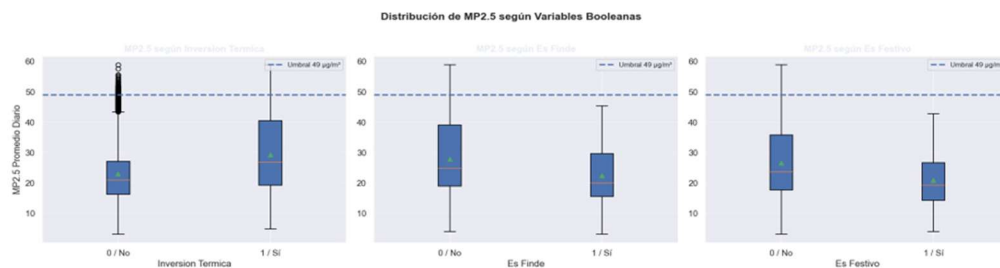


Figura 4. Distribución de MP2.5 según inversión térmica, fin de semana y festivo (fuente: SINCA, elaboración propia).

Visualización 3 — Tendencia temporal y estacionalidad. La evolución mensual por comuna evidencia un patrón estacional marcado: las concentraciones se elevan en los meses fríos (abril–agosto) y descienden en primavera-verano, con Cerro Navia, El Bosque y Pudahuel sistemáticamente sobre el resto, y Las Condes y Providencia en los niveles más bajos.

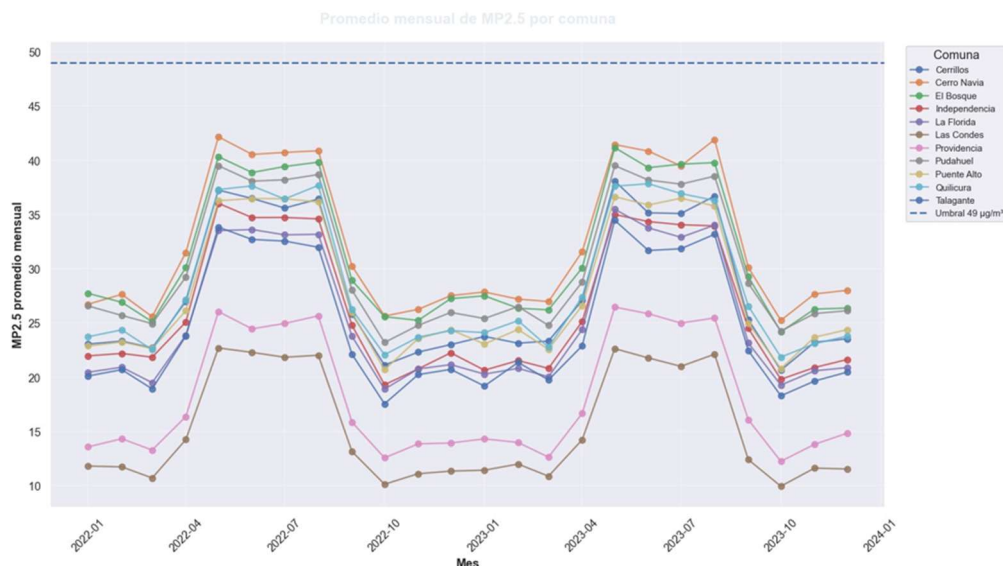


Figura 5. Promedio mensual de MP2.5 por comuna 2022–2023 (fuente: SINCA, elaboración propia).

VIII. Metodología del desarrollo técnico

El proyecto siguió un proceso iterativo a lo largo de las cuatro fases del ABP, articulando cada etapa con sus resultados. La metodología combinó programación estructurada (pipeline de preprocesamiento), recursiva (búsqueda binaria y merge sort) y orientada a objetos (jerarquía de analizadores), seleccionadas por su aporte a la modularidad, eficiencia y extensibilidad del sistema.

Fase	Foco	Aporte al resultado
F1	Definición y entorno reproducible	Estructura del repositorio, requirements.txt, entorno virtual y definición del problema.
F2	Preprocesamiento y transformación	Dataset consolidado y limpio (189.830 registros válidos), pipeline reproducible.
F3	Núcleo algorítmico, eficiencia y POO	Algoritmos recursivos, mediciones timeit, arquitectura POO extensible.
F4	Análisis, reproducibilidad y comunicación	Integración, visualizaciones finales, validación end-to-end y comunicación.

Tabla 8. Descripción de las fases F1–F4 y su relación con los resultados.

a. Validación y reflexión técnica

Las pruebas del sistema (sección V.c) confirmaron el correcto funcionamiento bajo casos normales, límite y excepciones. En comparación con las fases previas, el proyecto evolucionó desde funciones triviales de carga y limpieza (F1–F2) hacia un núcleo algorítmico con recursividad, vectorización y POO (F3–F4), incrementando la robustez, la eficiencia y la mantenibilidad. La reflexión grupal sobre escalabilidad confirma que la arquitectura admite nuevos contaminantes o estaciones sin reescritura del núcleo.

b. Resultados y Hallazgos del Análisis

Los resultados obtenidos permiten caracterizar el comportamiento del material particulado fino (MP2.5) en Santiago y validar las hipótesis exploratorias planteadas al inicio del proyecto.

Hallazgos principales

Se identificaron 8.579 horas críticas de MP2.5, equivalentes al 12,19 % del período analizado, mientras que MP10 registró solo 149 horas críticas (0,21 %), confirmando que MP2.5 constituye el contaminante atmosférico de mayor relevancia en la muestra estudiada.

El análisis temporal evidenció una estacionalidad marcada, observándose concentraciones más elevadas durante los meses fríos, comportamiento consistente con la literatura y con los episodios históricos de contaminación en la Región Metropolitana.

La temperatura promedio presentó la relación inversa más relevante con MP2.5 ($r = -0,54$), seguida por la radiación solar diaria ($r = -0,40$), indicando que condiciones atmosféricas frías y de baja radiación favorecen la acumulación de contaminantes.

Los análisis por condiciones contextuales mostraron que la inversión térmica incrementa en promedio $6,21 \mu\text{g}/\text{m}^3$ los niveles de MP2.5, constituyéndose como el factor meteorológico más asociado a episodios críticos.

Los boxplots y análisis comparativos evidenciaron diferencias significativas entre grupos de observación, confirmando que ciertas condiciones meteorológicas modifican la distribución y dispersión de las concentraciones registradas.

Interpretación de los resultados

Los hallazgos demuestran que el comportamiento del MP2.5 no es aleatorio, sino que responde a patrones temporales y meteorológicos identificables. La combinación de estacionalidad, correlaciones ambientales y episodios críticos permite justificar la construcción del núcleo algorítmico desarrollado en el Notebook 02, orientado a la detección automatizada de eventos de contaminación atmosférica.

c. Discusión — Limitaciones, Riesgos y Alcance

Los resultados obtenidos confirman la hipótesis inicial del proyecto: los episodios críticos de MP2.5 se concentran principalmente durante los meses fríos y se asocian a condiciones de estabilidad atmosférica caracterizadas por inversión térmica, baja temperatura y menor radiación solar. La identificación de 8.579 horas críticas de MP2.5 (12,19 % del período analizado) respalda la existencia de patrones ambientales consistentes y no aleatorios.

Limitaciones

El análisis se basa exclusivamente en las variables disponibles en el dataset SINCA, por lo que factores adicionales como emisiones vehiculares, actividad industrial, combustión residencial o variables topográficas no fueron incorporados.

La presencia de registros faltantes en algunas estaciones puede afectar la continuidad temporal de ciertas series y reducir la comparabilidad entre zonas geográficas.

El criterio de episodios críticos se construyó utilizando umbrales operativos derivados de la normativa ambiental, constituyendo una simplificación analítica del fenómeno real.

Riesgos de interpretación

Las correlaciones observadas entre MP2.5 y variables meteorológicas no implican causalidad directa. Por ejemplo, la relación inversa con temperatura ($r = -0,54$) y radiación ($r = -0,40$) debe interpretarse como asociación estadística y no como mecanismo causal demostrado.

La existencia de valores extremos puede influir en métricas basadas en promedios, por lo que los resultados deben interpretarse conjuntamente con medidas robustas como la mediana y los percentiles.

Los patrones identificados corresponden al período histórico analizado y podrían variar ante cambios estructurales en condiciones climáticas, urbanas o regulatorias futuras.

Alcance de las conclusiones

Los hallazgos poseen validez dentro del alcance temporal y geográfico del dataset estudiado. Los resultados permiten comprender el comportamiento del MP2.5 en Santiago, identificar condiciones asociadas a episodios críticos y justificar el desarrollo del núcleo algorítmico implementado en el Notebook

02. Sin embargo, la utilización de estos resultados para predicción operacional o apoyo a decisiones ambientales requiere modelos explicativos y predictivos complementarios.

d. Conclusiones — Aprendizajes y Trabajo Futuro

El Proyecto ABP permitió diseñar, implementar, validar y comunicar un sistema reproducible para el análisis de la calidad del aire en Santiago, integrando de forma consistente las distintas etapas del proceso de ciencia de datos. La utilización de Python, Pandas y NumPy facilitó la automatización del procesamiento y análisis exploratorio, mientras que la arquitectura modular desarrollada favoreció la mantenibilidad, trazabilidad y reutilización del código.

Desde la perspectiva analítica, los resultados confirmaron que el MP2.5 constituye el contaminante de mayor relevancia dentro del período estudiado, registrándose 8.579 horas críticas (12,19 % de la serie analizada). Asimismo, se observó una fuerte asociación entre los episodios críticos y condiciones meteorológicas específicas, particularmente inversión térmica, bajas temperaturas y menor radiación solar, evidenciando patrones temporales y estacionales consistentes.

Los hallazgos obtenidos validan la hipótesis inicial de que el comportamiento del MP2.5 responde a factores ambientales identificables y no a variaciones aleatorias. Esto permitió fundamentar el desarrollo del núcleo algorítmico implementado en el proyecto y demostrar la utilidad de las técnicas de análisis exploratorio para apoyar procesos de monitoreo ambiental.

Como líneas de trabajo futuro se propone incorporar modelos predictivos basados en Machine Learning, ampliar la cobertura geográfica mediante nuevas estaciones de monitoreo e integrar la actualización automatizada de datos desde la API de SINCA para evolucionar hacia una plataforma de análisis y predicción continua.

e. Trazabilidad de mejoras

La evolución técnica entre fases se documenta en el archivo changelog.md del repositorio, que registra para cada mejora la fecha, descripción, commit y justificación técnica. La siguiente tabla sintetiza las observaciones formativas y las acciones de mejora aplicadas:

Observación formativa	Acción de mejora	Evidencia (commit / PR)
Reforzar validaciones del núcleo	Incorporación de casos límite y manejo de excepciones (ValueError)	feat validación · PR #44
Outputs no legibles en GitHub	Rediseño GitHub-safe: colores sólidos, sin gradientes	fix · PR #71, #58
Numeración y formato de notebooks	Normalización de secciones y márgenes markdown	fix · PR #75, #76
Trazabilidad de contribuciones	Consolidación de identidades vía .mailmap	docs · PR #73, #74

Tabla 9. Trazabilidad de mejoras F1–F4 (detalle completo en changelog.md).

Síntesis del impacto. Las mejoras incrementaron la modularidad (de scripts a POO con herencia y polimorfismo), el rendimiento (algoritmos $O(\log n)$ y vectorización) y la documentación (notebooks con storytelling, README, informe institucional y changelog trazable).

IX. Presentación audiovisual

La presentación audiovisual del proyecto (5 a 8 minutos, alojada en Canvas Studio) comunica de forma técnica y profesional los procesos, resultados y conclusiones, según la siguiente estructura:

Apertura: equipo, nombre del proyecto y objetivo general.

Contexto: la problemática del MP2.5 en Santiago y su relevancia profesional.

Metodología: pipeline y herramientas, con evidencia visual o ejecución de una función del núcleo.

Resultados: las tres visualizaciones clave y los hallazgos algorítmicos.

Cierre: conclusiones, trabajo futuro y reflexión grupal sobre el aprendizaje.

Presentación visual

X. Bibliografía (APA 7)

- Las fuentes se presentan en orden alfabético, combinando documentación técnica oficial (≥ 2), bibliografía académica (≥ 1) y material docente (≥ 2).
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.^a ed.). MIT Press.
- García, S. (2025). Algoritmos recursivos y análisis de complejidad en Python [Material del curso]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- McKinney, W. (2022). Python for data analysis (3.^a ed.). O'Reilly Media.
- Ministerio del Medio Ambiente. (2024). Sistema de Información Nacional de Calidad del Aire (SINCA). <https://sinca.mma.gob.cl>
- Pérez, J. (2025). Programación orientada a objetos para ciencia de datos [Presentación de clase]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- Python Software Foundation. (2024). timeit — Measure execution time of small code snippets. <https://docs.python.org/3/library/timeit.html>
- The pandas development team. (2024). pandas documentation. <https://pandas.pydata.org/docs/>

Anexo A. Evidencia de control de versiones (F1–F4)

Este anexo reúne el panel completo de trazabilidad colaborativa del repositorio `abp_cienciadatos`, generado sobre el historial de Git con identidades consolidadas mediante `.mailmap`. Complementa la síntesis del apartado IV.b y constituye la evidencia objetiva de la contribución individual exigida por la rúbrica.

Para respaldar de forma auditable la gestión del control de versiones, se incorpora a continuación el panel de evidencia de trazabilidad colaborativa generado sobre el historial completo del repositorio **abp_cienciadatos**. Las cifras provienen de la consolidación de autores mediante `.mailmap` y reflejan la actividad real de los tres integrantes a lo largo de las cuatro fases del proyecto (F1–F4).



Figura A1. Dashboard ejecutivo F1–F4: visión consolidada de la trazabilidad del repositorio.

El **dashboard ejecutivo** sintetiza la salud global del repositorio en seis indicadores maestros: 226 commits 100% trazables, 81 Pull Requests históricos, 72 PR fusionados con éxito (merge rate de 88,9%), 9 PR cerrados sin merge —reemplazados por versiones corregidas—, 36 merges de integración y cero issues abiertos. El ciclo promedio de revisión de un PR fue de 12,9 minutos, evidenciando un flujo ágil. En conjunto, estos valores califican la salud del repositorio como **excelente**: un historial saludable, colaborativo y listo para la entrega final, con el 100% de los commits auditables.

2

MATRIZ EJECUTIVA POR FASE

Indicadores clave por fase del proyecto

Fase	Commits	PR Total	PR Mergeados	PR Cerrados Sin Merge	Merge Rate %	Ciclo PR prom. (min)
F1	41	12	10	2	83.3%	40.8
F2	39	6	5	1	83.3%	25.9
F3	81	26	24	2	92.3%	3.5
F4	25	7	7	0	100.0%	1.3
TOTAL	226	81	72	9	88.9%	12.9

88.9%

MERGE RATE
GENERAL

12.9 min

CICLO PR PROMEDIO
GENERAL

72

PR MERGEADOS
EXITOSAMENTE

100%

TRACKING Y
AUDITORIA

Figura A2. Matriz ejecutiva por fase: indicadores de gestión de Pull Requests en F1–F4.

La **matriz por fase** desglosa los indicadores anteriores a lo largo del ciclo de vida del proyecto y permite observar una clara **curva de madurez del equipo**. El merge rate progresa desde 83,3% en F1 y F2 hasta 92,3% en F3 y 100% en F4, mientras que el tiempo de ciclo de un PR se reduce drásticamente desde 40,8 minutos en F1 hasta apenas 1,3 minutos en F4. Esta tendencia demuestra que, conforme avanzó el proyecto, el equipo afinó su disciplina de ramas y revisiones, logrando integraciones cada vez más limpias y rápidas.

5

PARTICIPACIÓN POR INTEGRANTE · COMMITS

Distribución de commits por integrante



Contribución total del equipo: 226 commits auditable y trazables.

Figura A3. Participación por integrante: distribución de los 226 commits consolidados.

La **distribución de commits por integrante** documenta el aporte individual de cada miembro sobre la base consolidada: Rodrigo Chinchón Ayala concentra 166 commits (73,5%), Sergio Fernández Almonacid 33 (14,6%) y Pablo Villalobos González 27 (11,9%). Esta visualización cumple un rol clave para la rúbrica, al constituir **evidencia objetiva de la contribución individual** trazable hasta el historial de Git, garantizando transparencia en la participación del equipo.

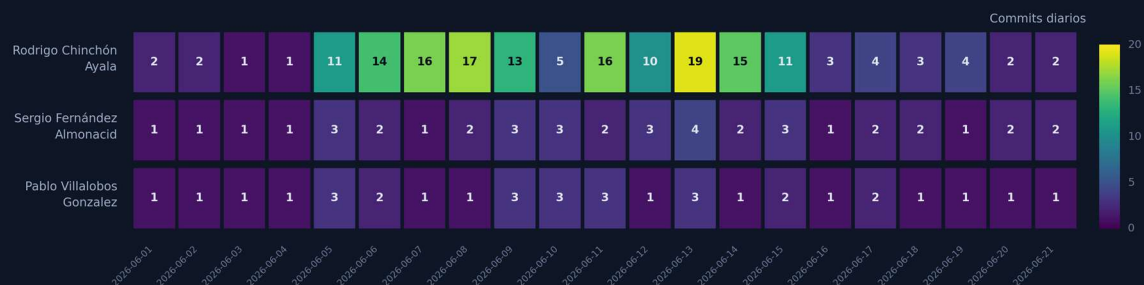


Figura A4. Evolución diaria de commits: actividad del repositorio a lo largo del período de trabajo.

La **serie temporal de actividad diaria** muestra el ritmo de trabajo del equipo a lo largo de 14 días activos, con un promedio de 3,9 commits por día y un pico de 19 commits en una sola jornada. El patrón revela un trabajo sostenido y con cadencia, sin grandes vacíos de actividad, lo que respalda la **regularidad del esfuerzo** frente a un modelo de última hora. Los valles puntuales coinciden con jornadas de revisión y consolidación más que de desarrollo activo.

8 MAPA DE CALOR · COMMITS POR INTEGRANTE Y DÍA

Agrupación normalizada de autores · valores diarios de commits



3

INTEGRANTES
normalizados

226

COMMITTS TOTALES
(base agrupada)

73.5%

MAYOR APORTE
Rodrigo Chinchón Ayala

19

PICO DIARIO
(commits en un día)

Nota técnica: autores consolidados mediante .mailmap; se eliminan alias duplicados y se mantienen valores diarios visibles solo donde existe actividad.

Figura A5. Mapa de calor de commits por integrante y día: intensidad de la colaboración.

El **mapa de calor** cruza las dos dimensiones anteriores —integrante y fecha— para revelar la intensidad de la colaboración día a día. La lectura confirma una participación simultánea y distribuida de los tres integrantes durante todo el período, sin zonas en blanco que sugieran inactividad de algún miembro. Los tonos más intensos se concentran en la franja central del calendario, coincidiendo con las fases de mayor carga técnica (F3), y validan el carácter **genuinamente colaborativo** del desarrollo.

9 MODELO DE TRAZABILIDAD COLABORATIVA

Flujo operativo evidenciado para reproducibilidad y control de cambios



RESULTADO



Repositorio con historial auditable, PRs identificables, fases separables y evidencia exportable para evaluación sumativa.



100%

Trazabilidad



Auditoría
Completa



Colaboración
Efectiva



Calidad de
Entrega



Listo para
Evaluación

Figura A6. Modelo de trazabilidad colaborativa: flujo operativo Branch → Commit → PR → Merge → Evidencia.

Finalmente, el **modelo de trazabilidad colaborativa** formaliza el flujo operativo que sustenta toda la evidencia anterior. Cada cambio recorre una secuencia disciplinada: se crea una **rama tipada** (feature/fix/docs), se registra mediante un **commit atómico**, se somete a revisión a través de un **Pull Request** trazable, se integra a la rama main mediante **merge** y queda respaldado como **evidencia exportable**. El resultado es un repositorio con historial auditable, PR identificables, fases separables y evidencia lista para la evaluación sumativa.

Anexo B. Arquitectura y estructura del proyecto

Este anexo reúne las vistas de arquitectura y estructura del proyecto que complementan los apartados IV.d y IV.e: el pipeline de la solución F4 (Figura B1) y el resumen ejecutivo de la estructura del repositorio por fases (Figura B2).



Figura B1. Arquitectura de la solución F4: pipeline de datos, análisis y resultados con su ecosistema tecnológico (elaboración propia).



Figura B2. Estructura del proyecto y resumen ejecutivo por fases F1–F4 (elaboración propia).